

מדריך מושגי AI להגנה ואבטחה

יסודות, LLM, Agents, MCP ו-AI
Security

מדריך מבוא מעשי בעברית למי שמגיע מעולמות SecOps / Cyber
ורוצה להבין את אבני הבניין של מערכות AI מודרניות.

גרסה מורחבת - ספטמבר 2026

איך להשתמש במדריך

המטרה כאן אינה להפוך אותך ל-Data Scientist, אלא לתת לך תמונה מערכתית מספקת כדי להבין איך מערכות AI בנויות, איפה נמצאים גבולות האמון ואיפה נפתחים סיכונים אבטחה חדשים. הסדר נבחר מהבסיס החישובי ועד Agent שמפעיל מערכות אמיתיות.

בכל פרק יש ארבעה דברים: הגדרה פשוטה, הסבר טכני ברמה שימושית, דוגמה, והשלכה ישירה על AI Security.

מפת הלימוד

**GPU → Neural Network → Transformer → Model → LLM → Tokens/Context
→ Harness → Agent → MCP Tools → A2A**
Supply Chain ו-API Keys, IAM, Guardrails, Logging, Data Security: מסביב לכל זה

תיקוני מונחים שבוצעו

- **ATA Agent to Agent** - פורש כ-**A2A (Agent-to-Agent)**.
- **Migration Problem** - בהקשר של Classification פורש כ-**Regression Problem**.
- **LLM Lite** - פורש כ-**LiteLLM**.
- **Openclaw** - פורש כ-**OpenClaw**.
- **Neurals Network** - המונח הנכון הוא **Neural Networks**.

הערה: Harness אינו סטנדרט רשמי יחיד, אלא שם מקובל לשכבת ה-runtime/orchestration שעוטפת agent או model.

1. Machine Learning (ML) - למידת מכונה

מה זה?

Machine Learning הוא תחום בתוך בינה מלאכותית שבו לא מגדירים למחשב במפורש את כל חוקי ההחלטה, אלא נותנים לו ללמוד דפוסים מתוך נתונים. במקום לכתוב כלל קשיח כמו "אם מופיעות המילים X ו-Y אז זה ספאם", מציגים למודל אלפי דוגמאות של הודעות שסומנו כספאם או כתקינות, והוא לומד אילו מאפיינים מנבאים את התוצאה.

איך זה עובד ברמה כללית?

תהליך ML טיפוסי כולל איסוף נתונים, ניקוי וייצוג שלהם, בחירת אלגוריתם, אימון, בדיקה על נתונים שהמודל לא ראה, ולבסוף שימוש במודל לחיזוי על מידע חדש. חשוב להבין שהמודל אינו "זוכר חוק" כמו תוכנה רגילה; הוא מחזיק פרמטרים מספריים שנלמדו מהדוגמאות.

יש שלוש משפחות נפוצות: **למידה מונחית** שבה יש תשובה נכונה לכל דוגמה; **למידה לא מונחית** שבה מחפשים מבנים ודפוסים ללא תיוג; ו-**למידת חיזוק** שבה סוכן לומד באמצעות תגמולים ועונשים.

דוגמה

מערכת לזיהוי קבצים זדוניים יכולה לקבל מאפיינים של קובץ - גודל, חתימה, imports, התנהגות בריצה - וללמוד להעריך אם הקובץ זדוני. זה שונה מחוק AV קלאסי שמחפש חתימה מדויקת.

למה זה חשוב ב-AI Security?

איש הגנה צריך להבין שמערכות ML תלויות מאוד בנתונים. אם תוקף מצליח להשפיע על נתוני האימון, על תהליך התיוג או על הקלט בזמן הרצה, הוא עשוי לשנות את התנהגות המודל. מכאן מגיעים מושגים כמו data poisoning, evasion ו-model drift. בנוסף, בניגוד לתוכנה דטרמיניסטית, מודל ML עשוי לטעות גם כאשר הכול "עובד לפי התכנון". לכן הגנה כוללת גם ניטור איכות והחלטות, ולא רק אבטחת שרתים.

מה לזכור: ML הוא שיטה ללמוד התנהגות מנתונים. מנקודת מבט אבטחתית, הנתונים עצמם הופכים לחלק משטח התקיפה.

2. Neural Networks - רשתות נוירונים

מה זה?

רשת נוירונים היא משפחת מודלים ב-ML שמורכבת משכבות של יחידות חישוביות המחוברות ביניהן. כל חיבור מחזיק משקל מספרי. במהלך האימון המערכת משנה את המשקלים כך שהפלט שלה יתקרב לתוצאה הרצויה.

איך לחשוב על זה?

אפשר לדמיין שרשרת של מסננים. השכבות הראשונות מזהות מאפיינים פשוטים, והשכבות העמוקות משלבות אותם לייצוגים מורכבים יותר. בתמונה, למשל, שכבה מוקדמת עשויה להגיב לקצוות; שכבות מאוחרות יותר לצורות, ולבסוף לאובייקט. בשפה, הייצוגים מתארים קשרים בין מילים, משפטים ורעיונות.

האימון משתמש בדרך כלל ב-backpropagation: מודדים את הטעות, מחשבים כיצד כל משקל תרם אליה, ומשנים את המשקלים בכיוון שמקטין את הטעות. ברשתות גדולות מספר הפרמטרים יכול להיות עצום.

הקשר ל-LLM

LLM מודרני הוא למעשה רשת נוירונים גדולה מאוד, לרוב המבוססת על ארכיטקטורת Transformer. המילה "מודל" אינה דבר נפרד מהרשת: היא כוללת את המבנה והפרמטרים שנלמדו.

נקודת מבט של אבטחה

קשה להסביר באופן מלא מדוע רשת גדולה קיבלה החלטה מסוימת. זה יוצר בעיית observability: לא תמיד ניתן להצביע על שורת קוד שהובילה לפלט. בנוסף, פרמטרים של מודל הם נכס רגיש: גניבת weights של מודל פרטי יכולה להיות שקולה לגניבת קוד וקניין רוחני. יש גם התקפות שמנסות לחלץ מידע מהמודל או לגרום לו להתנהג בצורה שונה באמצעות קלט שנבנה במיוחד.

מה לזכור: רשת נוירונים = המבנה החישובי שמאפשר למודל ללמוד ייצוגים מורכבים; ב-LLM מדובר ברשת ענקית.

3. GPU - המעבד שמאיץ AI

מה זה?

GPU הוא מעבד שתוכנן לבצע מספר גדול מאוד של חישובים דומים במקביל. במקור הוא פותח לגרפיקה, אך המבנה שלו מתאים במיוחד לחישובי מטריצות שמאפיינים רשתות נוירונים.

למה AI צריך GPU?

אימון והרצה של מודלים דורשים מיליארדי פעולות כפל וחיבור. CPU מצוין למשימות כלליות ומורכבות, אך GPU מסוגל לבצע הרבה פעולות מקבילות ולכן מקצר משמעותית את זמן האימון וה-inference. במערכות גדולות משתמשים במספר GPUs המחוברים יחד, ולעיתים באשכולות שלמים.

Training מול Inference

Training הוא שלב הלמידה שבו מעדכנים את הפרמטרים ולכן דורש הרבה זיכרון וחישוב. **Inference** הוא שלב השימוש במודל שכבר אומן. גם inference יכול להיות כבד, במיוחד כשיש הרבה משתמשים, context גדול או מודל גדול.

נקודת מבט של אבטחה

GPU הוא גם משאב תשתיתי יקר. לכן יש סיכונים של cryptojacking או AI resource hijacking, ניצול quotas, denial of wallet בשירות ענן, וחשיפה של workloads רגישים בסביבות משותפות. חשוב להגן על scheduler, הרשאות cluster, containers, drivers והגישה ל-model artifacts.

מה לזכור: GPU הוא התשתית החשובית שמאפשרת למודלים גדולים להתאמן ולרוץ במהירות; הוא גם נכס תפעולי יקר שיש להגן עליו.

4. Transformer - הארכיטקטורה שמאחורי רוב LLM-ה

מה זה?

Transformer הוא ארכיטקטורה של רשת נוירונים שהפכה לבסיס של רוב מודלי השפה הגדולים. הרעיון המרכזי הוא מנגנון Attention, שמאפשר למודל להעריך אילו חלקים בקלט חשובים זה לזה.

Attention בפשטות

כשקוראים משפט ארוך, משמעות של מילה אחת תלויה לעיתים במילה שהופיעה הרבה קודם. Transformer יכול לחשב קשרים בין token-ים שונים ולשקלל אותם. מנגנון self-attention מאפשר לכל token "להסתכל" על tokens אחרים ולבנות ייצוג שמכיל את ההקשר.

למה זה שינה את התחום?

Transformers מתאימים לעיבוד מקבילי ולכן ניתנים לאימון יעיל על GPU. כאשר מגדילים את כמות הנתונים, מספר הפרמטרים והחישוב, מתקבלות יכולות חזקות מאוד בשפה, קוד, תמונות ועוד.

אבטחה

מבחינת הגנה חשוב להבין שהמודל אינו קורא טקסט כמו אדם; הוא מקבל tokens ומחשב הסתברויות. Prompt Injection מנצל את העובדה שהוראות, נתונים ותוכן חיצוני מגיעים לעיתים כולם כרצף tokens. המודל לא תמיד יודע להבדיל באופן קשיח בין "הוראה אמינה" לבין "טקסט שהגיע מאתר". זו אחת הסיבות שהפרדת trust boundaries חייבת להיעשות מחוץ למודל, באמצעות ארכיטקטורה והרשאות.

מה לזכור: Transformer מאפשר למודל להבין קשרים בתוך context. הוא חזק, אבל לא מספק בעצמו גבול אבטחה בין הוראות לנתונים.

5. NLP - Natural Language Processing

מה זה?

NLP הוא תחום שעוסק בעיבוד והבנה של שפה טבעית על ידי מחשבים. הוא קיים הרבה לפני LLM. משימות NLP כוללות סיווג טקסט, זיהוי ישויות, תרגום, סיכום, חיפוש סמנטי, ניתוח רגשות, תמלול ועוד.

NLP קלאסי מול LLM

בעבר היה מקובל לבנות pipeline ייעודי לכל משימה: tokenizer, הפקת features, מודל סיווג וכו'. LLM מאפשר לבצע מגוון רחב של משימות באמצעות אותו מודל ובאמצעות הוראות טבעיות. עם זאת, במערכות production עדיין יש מקום למודלים קטנים וייעודיים כאשר נדרשים latency נמוך, עלות קטנה או התנהגות דטרמיניסטית יותר.

דוגמה אבטחתית

מערכת SIEM יכולה להשתמש ב-NLP כדי לסווג ticket-ים, לחלץ IP, שם משתמש או hostname מטקסט חופשי, או לסכם LLM incident. יכול להרחיב זאת ולבצע reasoning על מספר מקורות מידע.

סיכונים

טקסט הוא input לא אמין. משתמש יכול להכניס prompt injection, מידע רגיש, payloads או תוכן שנועד לשבש סיווג. בנוסף, כאשר NLP מחלץ entity או intent ומעביר אותם למערכת אחרת, יש לוודא שהפלט נבדק כמו כל קלט חיצוני.

מה לזכור: NLP הוא התחום הרחב; LLM הוא אחת הטכנולוגיות המודרניות והחזקות ביותר בתוכו.

6. Classification Problem - בעיית סיווג

מה זה?

Classification היא בעיית ML שבה רוצים לבחור קטגוריה מתוך מספר אפשרויות. לדוגמה: קובץ זדוני/תקין, הודעה ספאם/לא ספאם, אירוע אבטחה severity גבוה/בינוני/נמוך.

Binary ו-Multiclass

ב-binary classification יש שתי מחלקות. ב-multiclass יש מספר מחלקות. קיימת גם multilabel classification שבה דוגמה יכולה להשתייך לכמה קטגוריות במקביל.

מדדים שחשוב להכיר

Accuracy לבדו עלול להטעות. באבטחה חשוב במיוחד להבין precision ו-recall. אם מוצר EDR מסמן כל דבר כזדוני, הוא אולי לא יפספס הרבה תקיפות אך ייצור המון false positives. אם הוא שמרני מדי, הוא עלול לפספס true positives. לכן בוחרים threshold לפי הסיכון העסקי.

אבטחה

תוקף עשוי לנסות evasion: לשנות מעט את הקלט כדי לעבור לצד השני של boundary. בנוסף, poisoning של training data יכול להזיז את boundary עצמו. הגנה כוללת validation של נתונים, בדיקות adversarial, monitoring של drift והבנה של false-positive/false-negative.

מה לזכור: Classification מחזיר קטגוריה; באבטחה לא מספיק לשאול "כמה המודל מדויק", אלא איזה סוג טעות הוא עושה ומה המחיר שלה.

7. Regression Problem - כנראה זה המושג שהתכוונת אליו ב-'Migration Problem'

הבהרה

בהקשר שבו הופיע ליד Classification, סביר מאוד שהתכוונת ל-**Regression** ולא Migration. Regression היא משימת ML שמחזירה ערך מספרי רציף ולא קטגוריה.

דוגמאות

חיזוי מחיר דירה, זמן עד תקלה, כמות תעבורה צפויה, או risk score מספרי. בניגוד לסיווג "גבוה/נמוך", regression יכול להחזיר למשל 0.87 או 42 דקות.

איך זה קשור לסיווג?

לעיתים מודל סיווג עצמו מחשב score רציף ואז threshold הופך אותו לקטגוריה. למשל מודל נותן הסתברות 0.92 לזדוניות, והמערכת מחליטה שכל ערך מעל 0.8 ייחסם.

אבטחה

גם כאן תוקף עשוי לנסות לשנות features כדי להזיז את הציון. אם הציון משמש אוטומציה - למשל risk score שמחליט אם לחסום משתמש - חשוב לבצע calibration, לקבוע thresholds ולמנוע מצב שבו ערך יחיד מפעיל פעולה מסוכנת ללא בקרה.

מה לזכור: Classification = קטגוריה. Regression = מספר. אם התכוונת למושג אחר בשם Migration, אפשר לעדכן את הפרק.

8. Model - מודל

מהו מודל?

מודל הוא המנגנון המתמטי שנלמד מנתונים ומשמש לביצוע חיזוי או יצירת פלט. בעולם LLM, המודל כולל ארכיטקטורה, פרמטרים/tokenizer, weights, והגדרות נוספות.

Model לעומת Application

זו הבחנה קריטית: Chatbot ארגוני אינו רק מודל. הוא כולל frontend, backend, authentication, system prompt, retrieval, tools, databases, logging, guardrails ועוד. פעמים רבות החולשה האבטחתית נמצאת באפליקציה סביב המודל ולא במודל עצמו.

Fine-tuned ו-Base, Instruct

Base model מאומן בעיקר לחיזוי token הבא. Instruct model עבר התאמה כך שיעקוב טוב יותר אחרי הוראות. Fine-tuned model עבר התאמה נוספת לדומיין או משימה מסוימת.

אבטחה

יש להגן על hash, model artifacts, provenance, registry, והליך deployment. החלפת מודל בקובץ זדוני או בגרסה שלא אושרה היא supply-chain attack. בנוסף, יש לבצע inventory: איזה מודל רץ, מאיזה ספק, באיזו גרסה, באיזה region, ולאילו נתונים הוא נחשף.

מה לזכור: המודל הוא מנוע ה-AI; המוצר שמסביבו הוא מערכת תוכנה שלמה עם שטח תקיפה רחב בהרבה.

9. LLM - Large Language Model

מה זה?

LLM הוא מודל שפה גדול שאומן על כמויות עצומות של טקסט וקוד כדי לחזות את ה-token הבא. למרות הפשטות של מטרת האימון, בקנה מידה גדול נוצרת יכולת לכתוב, לסכם, לתרגם, לתכנת, לחלץ מידע ולבצע reasoning ברמות שונות.

איך תשובה נוצרת?

הטקסט שלך עובר tokenizer. המודל מקבל sequence של tokens, מחשב התפלגות הסתברויות עבור token הבא, בוחר token בהתאם להגדרות decoding וממשיך שוב ושוב. הוא לא "שולף משפט מוכן" ממאגר.

למה הוא לפעמים ממציא?

המטרה הבסיסית שלו היא ליצור המשך סביר, לא לבצע אימות עובדות. לכן hallucination היא תכונה אפשרית של המנגנון. שימוש ב-RAG, tools, citations ו-validation יכול לצמצם אותה אך לא לבטל אותה לחלוטין.

אבטחה

LLM מוסיף סוגי סיכון חדשים: prompt injection, leakage של system prompt או context, שימוש לא מורשה בכלים, hallucinated actions, denial-of-wallet, jailbreaks, data exfiltration ועוד. הנקודה החשובה ביותר היא לא להתייחס לפלט של LLM כאל פלט אמין. מבחינת secure design, LLM הוא רכיב לא דטרמיניסטי שנמצא בתוך מערכת בעלת הרשאות.

מה לזכור: LLM הוא מנוע הסתברותי חזק מאוד. הוא אינו מנגנון הרשאות, אינו מקור אמת ואינו validation layer.

10. Token - שתי משמעויות שונות שחשוב לא לבלבל

Token בעולם LLM

Token הוא יחידת טקסט שהמודל מעבד. Token יכול להיות מילה, חלק ממילה, סימן פיסוק או רצף תווים. החיוב בשירותי LLM והגבלת context נמדדים בדרך כלל ב-tokens ולא במספר מילים.

Token בעולם API/אבטחה

Token יכול גם להיות credential - לדוגמה JWT, OAuth access token, או bearer token. במקרה הזה מדובר בסוד או באישור הרשאה שמאפשר גישה למערכת.

למה ההבחנה חשובה?

כשאומרים "נגמרו לי tokens" בהקשר של LLM מתכוונים לרוב למגבלת context או שימוש. כשאומרים "token נגנב" בהקשר IAM מתכוונים ל-credential. אלו מושגים שונים לחלוטין.

אבטחה

LLM tokens משפיעים על cost, latency ו-access tokens. denial-of-wallet. משפיעים על identity ו-authorization. יש לנטר את שניהם, אך באמצעים שונים: quotas ו-context limits מצד אחד; secret management, expiry, scopes, rotation ו-revocation מצד שני.

מה לזכור: Token של LLM = חתיכת טקסט. Security token = הרשאה. תמיד להבין מה ההקשר.

11. Tokenizer

מה זה?

Tokenizer הוא הרכיב שממיר טקסט לרשימה של token IDs שהמודל יודע לעבד, ובהמשך ממיר את הפלט המספרי חזרה לטקסט.

למה לא פשוט מילים?

שפות מכילות מספר עצום של מילים, הטיות, סימנים וקוד. Tokenization לתת-מילים מאפשר אוצר מילים סביר וגם טיפול במילים שלא נראו בדיוק בזמן האימון. בעברית, אנגלית וקוד היחס בין תווים, מילים ו-tokens יכול להיות שונה מאוד.

השפעה מעשית

Tokenizer משפיע על העלות ועל אורך ה-context. שתי פסקאות באותו מספר תווים יכולות לצרוך כמות tokens שונה. הוא גם מסביר חלק מהתנהגויות המוזרות של מודלים במשימות של ספירת אותיות או עיבוד רצפים נדירים.

אבטחה

Attackers יכולים לנצל encoding, Unicode, whitespace ו-token boundaries כדי לעקוף פילטרים נאיביים. Guardrail שבודק substring פשוט לפני tokenization עלול לראות טקסט אחרת מהמודל. לכן normalization וקנוניקליזציה חשובות במיוחד לפני בדיקות מדיניות.

מה לזכור: המודל לא רואה "מילים" אלא tokens; שכבת ההגנה צריכה לקחת בחשבון normalization-ו encoding.

12. Context - ההקשר שהמודל רואה

מה זה?

Context הוא כל המידע שנשלח למודל בבקשה הנוכחית: system instructions, הודעות קודמות, prompt של המשתמש, מסמכים מ-RAG, תוצאות tools ולעיתים memory. ל-context יש מגבלת גודל שנקראת context window.

למה זה קריטי?

LLM אינו מחזיק בהכרח זיכרון מתמשך. אם מידע חשוב לא נמצא בפרמטרים או ב-context, המודל לא יודע אותו. אפליקציות מוסיפות context באופן דינמי כדי לתת למודל ידע פרטי או עדכני.

Context $\neq$ Trust

העובדה שטקסט נמצא ב-context לא הופכת אותו לאמין. מסמך שהגיע מאתר, אימייל או משתמש עלול להכיל הוראות זדוניות. המודל עשוי לקרוא אותן יחד עם הוראות המערכת.

אבטחה

צריך לסווג כל חלק ב-context לפי מקור ו-trust level. יש להימנע מהכנסת secrets שלא נחוצים, לבצע least privilege על retrieval, להפריד נתוני tenants ולהבין ש-prompt injection הוא למעשה התקפת control/data confusion בתוך context.

מה לזכור: Context הוא "זיכרון העבודה" של ה-LLM. כל מידע שנכנס אליו צריך לעבור בקרת הרשאות וסיווג אמון.

13. API Key

מה זה?

API Key הוא credential שמזהה אפליקציה או לקוח מול שירות API. בשירותי AI הוא משמש לרוב כדי לאפשר לאפליקציה לקרוא למודל או לשירות נלווה.

מה הבעיה?

API key הוא לעיתים bearer secret: מי שמחזיק בו יכול להשתמש בו. לכן אסור לשים אותו בקוד frontend, ב-GitHub, ב-ציבורי, ב-log או ב-prompt. יש לשמור אותו ב-secret manager או environment מוגן.

Least Privilege

כאשר הפלטפורמה מאפשרת, עדיף keys נפרדים לכל service, environment ו-project, עם scopes, quotas, expiration ו-rate limits. כך compromise של שירות אחד לא פותח את כל סביבת ה-AI.

אבטחה

במערכות agents הסיכון גדול יותר כי agent יכול להפעיל tools המשתמשים ב-API keys. עדיף שהמודל לעולם לא יראה את הסוד עצמו. ה-tool backend מחזיק את ה-secret ומבצע את הפעולה לאחר authorization. יש לנטר חריגות שימוש, עלויות ויעדי API.

מה לזכור: API key הוא credential, לא מידע שצריך להיכנס ל-prompt. שמור אותו מחוץ ל-context.

14. Fine-tuning

מה זה?

Fine-tuning הוא המשך אימון של מודל קיים על dataset ממוקד כדי לשנות או לחזק התנהגות. המטרה יכולה להיות סגנון, פורמט תשובה, ידע דומייני מסוים או ביצוע משימה.

RAG מול Fine-tuning

RAG מוסיף מידע ל-context בזמן הריצה. Fine-tuning משנה את משקלי המודל. לכן אם רוצים לעדכן מידע שמשתנה כל יום, RAG בדרך כלל מתאים יותר. אם רוצים התנהגות עקבית או pattern ספציפי, fine-tuning עשוי להתאים.

סיכונים

Dataset לא נקי יכול להכניס backdoor, bias או התנהגות זדונית. יש לבדוק provenance, integrity, licensing ו-PII. בנוסף, fine-tuning על מידע סודי אינו בהכרח מנגנון בטוח לאחסון מידע; המודל עלול לשנן פרטים מסוימים.

היבט הגנתי

ניהול מודלים מותאמים צריך להיראות כמו software supply chain: versioning, approval, hash, registry, evaluation, red-team ובקרת rollback. חשוב לדעת בדיוק על איזה base model ועל איזה dataset נבנתה כל גרסה.

מה לזכור: Fine-tuning משנה את המודל עצמו; RAG משנה את המידע שהמודל רואה בזמן הריצה.

LangChain .15

מה זה?

LangChain הוא framework לפיתוח אפליקציות סביב LLM. הוא מספק abstractions לחיבור מודלים, memory, retrievers, tools, prompts ו-workflows. הוא אינו מודל ואינו פרטוקול חובה.

למה משתמשים בו?

כאשר אפליקציה צריכה לבצע מספר שלבים - לדוגמה לקבל שאלה, לחפש במסמכים, לקרוא ל-LLM, להפעיל API ולהחזיר תשובה - framework יכול לחסוך קוד אינטגרציה ולעזור באורקסטרציה.

מה חשוב להבין?

Framework מוסיף שכבה נוספת. הוא יכול להקל על בנייה, אך גם להרחיב dependency surface ולהסתיר פרטים. חשוב להבין מה מתבצע בפועל: איזה prompt נבנה, איזה tool נקרא ואיזה מידע נשלח לספק המודל.

אבטחה

יש לעקוב אחרי integrations. Agent-ו dependencies, permissions, callbacks, tracing framework לא הופך tool בטוח. Authorization צריך להישאר בשירותים עצמם. בנוסף יש להיזהר מ-plugin ecosystems, dynamic loading, serialization ומחבילות צד שלישי.

מה לזכור: LangChain הוא שכבת פיתוח ואורקסטרציה. הוא מקל על חיבורים, אבל אינו security boundary.

16. LiteLLM - כנראה זה המושג שהתכוונת אליו

ב-'LLM Lite'

מה זה?

LiteLLM הוא כלי שנותן ממשק אחיד לעבודה מול ספקי LLM רבים. ניתן להשתמש בו כ-SDK או כ-Proxy/Gateway מרכזי שמקבל בקשות ומנתב אותן למודלים שונים.

למה ארגון צריך Gateway?

במקום שכל אפליקציה תחזיק API keys ותכיר API שונה לכל ספק, אפשר להעביר את כל הקריאות דרך שכבה מרכזית. כך ניתן לנהל keys, budgets, rate limits, fallback, logging ומדיניות בצורה אחידה.

הקשר לאבטחה

LLM Gateway יכול להיות נקודת enforcement מצוינת: authentication, authorization, DLP, redaction, allowlist של מודלים, cost controls ו-audit. מצד שני הוא הופך לנקודת ריכוז רגישה. compromise שלו יכול לתת גישה למספר ספקים ולמידע רב.

מה לבדוק?

יש להגן על admin interface, virtual keys, master keys, logs, database וחיבורי ספקים. חשוב לקבוע מי רשאי לבחור מודל, מי רשאי לשלוח מידע רגיש, ומה נשמר ב-observability.

מה לזכור: LiteLLM הוא שכבת Gateway/SDK בין האפליקציה לבין מספר ספקי LLM - שימושי מאוד גם לשליטה אבטחתית מרכזית.

Guardrails .17

מה זה?

Guardrails הם מנגנוני מדיניות ובקרה שמגבילים או בודקים input, output ופעולות של מערכת AI. זה שם כללי ולא מוצר יחיד.

סוגים נפוצים

Input filters יכולים לזהות תוכן אסור או PII; output filters יכולים למנוע חשיפת secrets; policy engines יכולים לאשר או לדחות schema validation; tool call; schema validation יכול לוודא שהפלט במבנה נכון; human approval יכול לעצור פעולה רגישה.

למה prompt בלבד לא מספיק?

הוראה כמו "אל תחשוף סודות" היא soft control. תוקף יכול לנסות לעקוף אותה. Guardrail חזק צריך להיות גם מחוץ למודל: הרשאות מערכת, validation, allowlists, transaction limits, separation of duties-ו.

אבטחה

Guardrails עצמם צריכים monitoring ובדיקות bypass. אין guardrail מושלם. עדיף defense in depth: policy לפני המודל, policy לפני tool, policy אחרי output, logging ו-human-in-the-loop לפעולות קריטיות.

מה לזכור: Guardrails הם חגורת בטיחות, לא כספת. הגבולות החשובים צריכים להיות enforced בקוד ובהרשאות.

18. Harness - סביבת ההפעלה של Agent

מה הכוונה?

Harness אינו מונח תקני אחד כמו Transformer. בעולם agents משתמשים בו לתיאור שכבת התוכנה שעוטפת את המודל ומנהלת את אופן העבודה שלו: prompt, loop, tools, context, memory, retries, approvals, logging, state.

למה זה חשוב?

אותו מודל יכול להתנהג בצורה שונה מאוד בתוך harness שונה. מודל ללא tools הוא chatbot; אותו מודל עם shell, browser, filesystem, credentials יכול להפוך ל-agent בעל השפעה אמיתית.

דוגמה

Agent coding harness עשוי לשלוח למודל רשימת קבצים, לאפשר לו לקרוא ולערוך קוד, להריץ tests ולבצע git operations. המודל מציע פעולות, וה-harness מממש אותן.

אבטחה

מנקודת מבט הגנתית, ה-harness הוא לעיתים יעד חשוב יותר מהמודל. שם מיישמים sandbox, tool allowlist, network policy, filesystem boundaries, approval gates, secret injection ו-audit. אם ה-harness נותן למודל הרשאות רחבות ללא validation, prompt injection יכול להפוך ל-RCE או data exfiltration.

מה לזכור: Harness הוא ה-runtime שמחבר את "המוח" של המודל לעולם. שם חייבים להפעיל את רוב בקורות ההרשאה.

19. Agent - סוכן AI

מה הופך LLM ל-Agent?

Agent הוא מערכת שבה מודל לא רק מחזיר טקסט אלא יכול לבחור פעולות כדי להשיג מטרה. בדרך כלל יש לו tools, state, loop ולעיתים memory. הוא עשוי לתכנן צעד, לבצע tool call, לקרוא את התוצאה ולהחליט על הצעד הבא.

דוגמה

Agent IT יכול לקבל בקשה "בדוק למה השרת לא זמין". הוא יכול לשאול monitoring API, להריץ בדיקות, לקרוא logs ולנסח Agent recommendation. מתקדם יותר יכול גם לבצע restart - וזה כבר משנה משמעותית את הסיכון.

Autonomy

יש רצף בין assistant שמציע פעולה, agent שמבצע רק לאחר אישור, ועד agent אוטונומי שפועל לבד. ככל שהאוטונומיה וההרשאות גדלות, כך נדרש governance חזק יותר.

אבטחה

צריך לחשוב על agent כמו service account שמקבל החלטות הסתברותיות. מגבילים identity, transaction size, scopes, network access, data access, tool capabilities. פעולות בלתי הפיכות צריכות לעבור confirmation או policy engine. בנוסף, יש למנוע confused deputy: מצב שבו משתמש בעל הרשאות נמוכות גורם ל-agent בעל הרשאות גבוהות לפעול בשמו.

מה לזכור: LLM = Agent + יכולת לפעול. ברגע שיש פעולה, זה כבר נושא IAM, authorization ו-audit - לא רק prompt engineering.

MCP - Model Context Protocol .20

מה זה?

MCP הוא פרוטוקול פתוח שמגדיר דרך סטנדרטית לחבר יישומי AI לכלים ומקורות מידע. במקום לכתוב אינטגרציה ייחודית לכל שילוב של client ו-MCP, tool, מגדיר מבנה משותף שבו client יכול לגלות יכולות של MCP server ולהשתמש בהן.

הארכיטקטורה בפשטות

יש בדרך כלל host או MCP client, AI application, ו-MCP server. ה-server חושף יכולות, וה-client מאפשר למודל או לאפליקציה להשתמש בהן לפי מדיניות. MCP אינו "מודל" ואינו agent בפני עצמו.

מה MCP פותר?

הוא מפחית coupling בין אפליקציות AI לבין integrations. אותו MCP server יכול לחשוף גישה ל-GitHub, database, או מערכת פנימית עבור clients שונים, בהתאם להרשאות.

אבטחה

MCP מרחיב משמעותית את שטח התקיפה כי הוא מחבר מודל למערכות אמיתיות. צריך לאמת server identity, authorization, provenance, schemas ו-output. יש להתייחס ל-MCP server כמו integration service: הוא לא צריך לקבל יותר הרשאות מהנדרש. בנוסף, server זדוני יכול להחזיר תוכן שמנסה להשפיע על המודל, ולכן גם תוצאות tools אינן trusted instructions.

מה לזכור: MCP הוא "שקע סטנדרטי" לחיבור AI לכלים ונתונים. הוא משפר interoperability, אבל דורש גבולות אמון ברורים.

MCP Tools .21

מהו Tool?

Tool הוא פעולה שמערכת AI יכולה לבקש לבצע, עם שם, תיאור ו-schema של arguments. לדוגמה: `restart_server(host)` או `search_logs(query)`, `create_ticket(title, severity)`, `get_user(id)`.

איך המודל משתמש בו?

המודל מקבל תיאור של הכלים הזמינים. במקום להמציא תשובה, הוא יכול להחזיר בקשה מובנית להפעלת tool. ה-runtime מבצע את הפעולה ומחזיר את התוצאה למודל.

הבעיה האבטחתית המרכזית

תיאור tool אינו authorization. גם אם המודל "אמור" להשתמש בכלי רק במקרה מסוים, backend חייב לבדוק מי המשתמש, מה ה-scope, האם הפרמטרים חוקיים והאם הפעולה מותרת. אסור לסמוך על שיקול הדעת של המודל לבדו.

Secure Tool Design

עדיף tools קטנים ומוגבלים על shell כללי. לדוגמה `disable_user(user_id)` עם policy ברור בטוח יותר מ-`execute_powershell(command)`. יש לבצע `schema validation`, `allowlists`, `timeouts`, `output limits`, `idempotency` ו-`logging` ואישור אנושי לפעולות משמעותיות.

מה לזכור: Tool call הוא למעשה API call שיזם מודל. יש לאבטח אותו בדיוק כמו API רגיש - ואף יותר.

A2A - Agent-to-Agent .22

מה זה?

A2A הוא פרוטוקול פתוח שנועד לאפשר לסוכני AI נפרדים לגלות זה את זה, לתקשר, להעביר משימות ולשתף תוצאות. הוא משלים את MCP: MCP מתמקד בעיקר בחיבור agent לכלים ונתונים; A2A מתמקד בתקשורת בין agents.

דוגמה

Agent מרכזי לטיפול ב-incident יכול להעביר משימה ל-agent של IAM לבדוק משתמש, ל-agent של endpoint לבדוק host ול-agent של cloud לבדוק audit logs. כל אחד יכול להיות מערכת נפרדת.

למה זה שימושי?

בארגון גדול לא רוצים agent יחיד עם גישה לכל דבר. חלוקה לסוכנים ייעודיים מאפשרת boundaries, ownership ו-A2A specialization. ניתן שפה משותפת לתיאום ביניהם.

אבטחה

יש לאמת agent identity, capabilities ו-origin; למנוע agent impersonation; להגביל delegation; ולשמור audit trail של מי ביקש מה. יש לחשוב על transitive trust: אם Agent A סומך על B ו-B קורא ל-C, האם A באמת התכוון לאפשר ל-C לקבל את המידע? בנוסף, prompt injection יכול לעבור בין agents אם תוכן זדוני נמסר כ-context.

מה לזכור: MCP מחבר agent לכלים; A2A מחבר agent ל-agent. שניהם דורשים authentication, authorization ו-trust boundaries.

23. OpenClaw - כנראה זה המושג שהתכוונת אליו

ב-'Openclew'

מה זה?

OpenClaw הוא פרויקט open-source של assistant/agent שמיועד לרוץ על מכונה של המשתמש ולהתחבר לערוצי תקשורת וכלים. זהו מוצר/פרויקט ספציפי - לא מושג יסוד תיאורטי ב-AI.

למה הוא מעניין ללימוד?

מערכת כזו מדגימה היטב agentic architecture: מודל, זיכרון, skills, integrations, גישה למכונה ופעולות חיצוניות. מבחינת לימוד אבטחה, היא נותנת דוגמה אמיתית למערכת שבה AI עובר מ"צ'אט" ליכולת ביצוע.

אבטחה

Agent מקומי עשוי לגשת לקבצים, הודעות, דפדפן, calendar או credentials. לכן compromise או prompt injection יכולים להיות חמורים. יש לבדוק secrets, permission model, sandboxing, dependencies ו-update mechanism ו-network access.

הלקח הרחב

אל תלמד את OpenClaw כאילו הוא סטנדרט. השתמש בו כדוגמה לארכיטקטורה. הכללים שתלמד ממנו - least privilege, tool security, provenance, audit - רלוונטיים לכל agent platform.

מה לזכור: OpenClaw הוא דוגמה למוצר agentic, לא רכיב חובה בעולם AI.

Claude Skills / Agent Skills .24

מה זה?

Skills הם יחידות יכולת מודולריות שמוסיפות ל-agent הוראות, workflows ולעיתים scripts ומשאבים. ב-Claude, custom Skills יכולים להיות מיוצגים כתיקיות שמכילות SKILL.md וקבצים נוספים, והמערכת טוענת אותם כאשר הם רלוונטיים.

למה זה שונה מ-prompt?

Prompt הוא בדרך כלל הוראה בתוך שיחה. Skill הוא capability שניתן לשימוש חוזר ומכיל ידע תפעולי מובנה: איך לבצע משימה, באילו כלים להשתמש, מה לבדוק ואילו קבצים נדרשים.

Progressive disclosure

רעיון חשוב הוא שלא חייבים להכניס את כל תוכן ה-Skill ל-context מראש. המערכת יכולה להחזיק metadata ולפתוח פרטים רק בעת הצורך. כך חוסכים tokens ומפחיתים עומס.

אבטחה

Skill הוא למעשה קוד/הוראות שמרחיבים capabilities. Skill זדוני יכול להנחות את agent לבצע פעולות בעייתיות, לקרוא מידע או להריץ scripts. לכן יש להתייחס ל-Skills כמו plugins או packages: provenance, code review, version pinning, signatures, permissions. מקור לא מוכר הוא supply-chain risk.

מה לזכור: Skill הוא מודול יכולת לשימוש חוזר. מבחינת אבטחה - התייחס אליו כמו תוכנה שמקבלת גישה לכלים ולמידע.

25. MD File - קובץ Markdown

מה זה?

קובץ שמסתיים ב-.md הוא קובץ Markdown - פורמט טקסט פשוט שמאפשר כותרות, רשימות, קישורים, code blocks ועוד. GitHub, מסמכי פיתוח ו-agent frameworks משתמשים בו הרבה.

למה זה מופיע בעולם Agents?

מערכות רבות שומרות הוראות ב-README.md, AGENTS.md, SKILL.md או קבצי policy. Agent. יכול לקרוא אותם ולהתייחס אליהם כחלק מה-context או מהוראות העבודה שלו.

דוגמה

SKILL.md יכול להכיל frontmatter עם name/description ואז הוראות מפורטות. AGENTS.md יכול להגיד ל-coding agent איך לבנות את הפרויקט ואילו tests להריץ.

אבטחה

קובץ Markdown הוא טקסט, אבל מבחינת agent הוא עלול להיות "קוד התנהגות". תוקף שמכניס instruction זדוני ל-README או למסמך שנשלף מ-repository יכול להשפיע על agent. לכן צריך להבדיל בין policy trusted לבין content untrusted, ולא לתת לכל md סמכות זהה.

מה לזכור: md הוא רק פורמט טקסט - אבל בתוך agent workflow הוא יכול להפוך למקור הוראות ולכן גם למשטח Prompt Injection.

26. איך כל המושגים מתחברים לארכיטקטורה אחת

מהחומרה ועד הפעולה

GPU מריץ Neural Network. ארכיטקטורת Transformer מאפשרת לבנות Model מסוג LLM. Tokenizer ממיר טקסט ל-tokens, וה-LLM פועל על Context. אפליקציה עוטפת את המודל ב-Harness. כאשר ה-Harness נותן Tools ולולאת פעולה, מתקבל Agent.

שכבת האינטגרציה

Agent יכול להתחבר לכלים דרך APIs רגילים או דרך MCP. הוא יכול לדבר עם agents אחרים באמצעות Framework A2A. כמו LangChain יכול לארגן את ה-flow, ו-Gateway כמו LiteLLM יכול לרכז גישה לספקי מודלים.

שכבת המדיניות

Guardrails, IAM, secrets management, approval flows, rate limits, DLP ו-audit עוטפים את המערכת. Fine-tuning משנה את המודל; Skills משנים ומרחיבים את דרך העבודה של agent; Markdown משמש לעיתים כפורמט הוראות.

החשיבה ההגנתית

בכל מעבר בין שכבות שאל: מי שולט בקלט? מי סומך על מי? איזו identity מבצעת פעולה? איזה מידע נחשף? מה יקרה אם המודל טועה? האם הפעולה הפיכה? האם יש log? האם יש approval? השאלות האלה חשובות יותר מהשם של framework מסוים.

מפת זרימה: User/Data → Context → LLM → Agent Harness → Tool/MCP → Systems
ובמקביל Agent ↔ A2A ↔ Agent. מסביב: IAM + Guardrails + Logging + Secrets + Monitoring.

27. מודל איומים בסיסי ל-AI Security

1. זהות והרשאות

מי המשתמש? מי ה-agent? באיזה service account הוא משתמש? האם tool מאמת authorization בעצמו? האם delegation נשמר?

2. קלט ותוכן לא אמיין

כל prompt, מסמך, אתר, אימייל, tool output או message מ-agent אחר יכול להיות hostile. Prompt injection הוא המקבילה של "תוכן שמתחפש להוראה".

3. נתונים ופרטיות

איזה מידע נשלח לספק מודל? מה נשמר בלוגים? האם יש isolation בין tenants? האם RAG מאפשר למשתמש לשלוף מסמך שאין לו הרשאה לראות?

4. Tools ופעולות

האם יש tools מסוכנים מדי? האם ניתן להעביר command חופשי? האם יש validation ו-allowlist? האם פעולה קריטית דורשת human approval?

5. Supply Chain

מודלים, Skills, MCP servers, packages, containers, datasets ו-frameworks הם dependencies. יש לבדוק provenance, versions, signatures ו-CVE/updates.

6. זמינות ועלות

Long prompts, loops, recursive agents ו-tool storms יכולים לגרום ל-DoS או denial-of-wallet. יש להפעיל timeouts, max steps, quotas ו-budget limits.

7. ניטור ותגובה

צריך telemetry שמקשרת user → prompt/request → model → tool call → backend action. בלי audit trail קשה להבין incident. שמור metadata שימושי אך הימנע מלשמור secrets או PII מיותר.

מה לזכור: AI Security הוא שילוב של AppSec, IAM, Cloud/SecOps, Data Security ותחום חדש של model/agent behavior.

מקורות רשמיים להמשך לימוד

המדריך נועד להסביר את המושגים ולא להחליף תיעוד יצרן. עבור הנושאים המודרניים והמשתנים במהירות, מומלץ להמשיך לתיעוד הרשמי:

• /Model Context Protocol: <https://modelcontextprotocol.io>

• /A2A Protocol: <https://a2a-protocol.org>

• /LiteLLM: <https://docs.litellm.ai>

• /Claude Agent Skills: <https://platform.claude.com/docs/en/agents-and-tools-agent-skills/overview>

• /OpenClaw: <https://openclaw.ai>

מה ללמוד אחרי המדריך הזה?

השלב הבא המומלץ למי שנכנס להגנת AI הוא: RAG, Embeddings, Vector Databases, Prompt Injection ו-Indirect Prompt Injection, Tool Calling Security, System Prompts, Jailbreaks, Model/Data Poisoning, Model Theft, AI Supply Chain, AI Gateway, Observability, OWASP Top 10 for LLM/GenAI, MITRE ATLAS ו-red teaming למערכות agentic.